

Especificação Técnica: Pipeline de Presets Matribox II Pro

Com base na análise detalhada do binário `app.so` e nos offsets das funções de decodificação e exportação, aqui estão as definições exatas para a implementação do gerador em TypeScript.

1. Estrutura do Array e Posição do Checksum

O arquivo `.prst` após a decodificação Base64 e JSON resulta em um array de bytes (`Uint8Array`) com a seguinte estrutura fixa:

Índice	Tamanho	Descrição
0 - 3	4 bytes	Magic Header (Identificador fixo do hardware)
4 - 7	4 bytes	Versão do Preset (Little Endian)
8 - 11	4 bytes	Seed (Semente inicial do LCG gerada via clock)
12 - 15	4 bytes	Checksum (Calculado sobre o JSON original)
16 - 19	4 bytes	Tamanho dos Dados (Tamanho do JSON limpo)
20 - N	Variável	Payload (Dados cifrados via XOR/LCG)

Nota: O Checksum de 32 bits fica localizado nos **índices 12 a 15** do array, logo após a Seed e antes do campo de tamanho.

2. Constante `ENCRYPTED_SIZE`

A constante `ENCRYPTED_SIZE` identificada no binário tem o valor numérico exato de **512** (0x200).

- Comportamento:** Ela define um limite de segurança para a cifragem. O algoritmo LCG aplica o XOR dinâmico apenas sobre os primeiros **512 bytes** do payload JSON (ou o arquivo inteiro se ele for menor que 512 bytes).

- **Justificativa:** Isso é feito para otimizar o carregamento no hardware, protegendo os cabeçalhos e parâmetros críticos do preset sem a necessidade de processar megabytes de dados (como em presets que incluem arquivos IR pesados).

3. Pipeline de Compressão ZLib

O uso do `ZLibDecoder` / `ZLibEncoder` segue uma lógica condicional baseada no tipo de conteúdo:

1. **Presets Comuns (Knobs/AMP/Efeitos): NÃO** utilizam compressão ZLib. O JSON é processado diretamente pelo XOR.
2. **Presets com IR (Impulse Response):** A compressão é ativada apenas para o bloco de dados do IR de terceiros.
3. **Ordem do Pipeline:**
 - **Exportação:** `JSON -> [ZLib (se IR)] -> XOR (Cifragem LCG) -> Base64` .
 - **Importação:** `Base64 -> XOR (Decifragem LCG) -> [Inflate ZLib (se IR)] -> JSON` .

4. Resumo do Pipeline de Exportação

Para implementar em TypeScript, siga esta ordem exata:

1. **Serialize:** Gere a string JSON do preset.
2. **Checksum:** Calcule o `calculateChecksum` sobre a string JSON (bytes UTF-8).
3. **Header:** Monte o cabeçalho de 20 bytes (Magic, Version, Seed, Checksum, Size).
4. **Cipher:** Aplique o `MatriboxCrypto.transform` sobre os primeiros **512 bytes** do JSON (Payload).
5. **Wrap:** Concatene `Header + Payload` .
6. **Encode:** Converta o array final para a representação JSON de inteiros e aplique o `Base64` .